

Generators

Generators are an easy way to speed up your work. You can simply run a command rather than opening a directory, creating a new file and then saving it. Simply open the terminal, reach into your project directory and run the *php artisan* command to look for help regarding generators. If you don't know what parameters to pass with each command, just type *php artisan help <your generator option>*. Here are a few generators:

```
make:model - Create an eloquent model class
make:provider - Create a new service provider class
make:event - Create a new event class
make:controller - Create a new resource controller class
make:command - Create a new command class
make:migration - Create a new migration file
make:middleware - Create a new middleware class
make:request - Create a new form request class
make:console - Create a new artisan command
```

Blade

Blade is a template engine for Laravel. There isn't much improvement here, though. In older versions, we had the `{{ }}` and `{{{ }}` brackets - the double curly brackets were used for echo and the triple curly brackets escaped the variable. Now, in the newer version, both `{{ }}` and `{{{ }}` are used to escape the variable, and we now use `{!! !!}` for echo. Common usage is as follows: `{!! $var !!}`

Contracts

Contracts are interface classes used to provide core services to your application. For example, the Mailer contract defines the methods required for sending email. The reason behind the use of an interface is to achieve loose coupling and simplicity. When tightly coupled implementation is written by the user and if any change occurs in the API, mostly in vendor-specific cases, code must change as well. Similarly, if we want to replace the underlying package with another package, we will have to modify the repository. This can be understood by looking at a situation in which we want to use the package name *cache*; so instead of using a cache class, we can use the *Illuminate\Contracts\Cache* interface, and you are free to use the functionality without changing much of the code.

Helpers

A few helper functions have been replaced in Laravel 5. For instance, in Laravel 4, if we want to render a view, we can use:

```
view::make ('VIEW_NAME');
```

In the new version, it is simplified to...

```
view ('VIEW_NAME')
```

Other helper functions are: *response ()*, *redirect ()*, *action ()*, *abort ()*, *back ()*, *with ()*, *dd ()*, *url ()*, *Routing - get ()*, *post ()*, *delete ()* and many more.

Authentication

Authentication in Laravel 5 is very simple, as everything is configured out-of-the-box. The configuration file is located at *config/auth.php*, which contains various options. Laravel includes the *App\User* model in the project's root directory, by default. Here, two new features have been introduced —authenticating and registering new users, and password resetting.

Middleware

Middleware is a layer attached to HTTP routes for the filtering mechanism—for instance, when we want to allow only some pages to be viewed by guest users of a Web application while important pages can be viewed by registered users. Such verification can be done by middleware. To see the list of registered middleware, open the *kernel.php* file which can be found at *app\Http*. To create new middleware, type the command *php artisan make:middleware <middleware name>* from the root directory. We can register new middleware by appending *\$middleware* array, and new route middleware can be registered by appending *\$routeMiddleware* array. Let me give you a simple example. If we see the *app\Http\Middleware\Authenticate.php* file, which works out-of-the-box with Laravel, you can see the code written to check if the user is authenticated or not. If we want to use that middleware to check if the user is logged in, only then is the welcome page served. Let's suppose the *WelcomeController.php* file is responsible for serving the request. Code (a) shows how we can add middleware to our controller. The same can also be done by adding middleware to the *routes.php* file instead of the controller. See Code (b) for the same.

<pre>class WelcomeController extends Controller { public function __construct() { \$this->middleware('auth'); } }</pre>	<pre>Route::get('welcome', ['middleware' => 'auth', 'uses' => 'WelcomeController@ welcome']);</pre>
(a)	(b)

Elixir

Elixir provides an API for our application's basic Gulp tasks. For asset management, we use Elixir and Gulp. The Gulp package resides in the *package.json* file under the